

## Context

- Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.
- This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.
- By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

The data is available in 8 different csv files:

1. customers.csv
2. geolocation.csv
3. order\_items.csv
4. payments.csv
5. reviews.csv
6. orders.csv
7. products.csv
8. sellers.csv

## Problem Statement :

**I. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset.**

**A. Data type of all columns in the “customers” table.**

```
SELECT DISTINCT
    column_name,
    data_type
FROM
    `delta-trees-469521-c4`.Target.`INFORMATION_SCHEMA.COLUMNS`
WHERE
```

```
table_catalog = 'delta-trees-469521-c4' AND table_schema = 'Target' AND
table_name = 'customer_purchase';
```

Query completed

Query results

Save results Open in

| Job information | Results                  | Visualisation | JSON | Execution details | Execution graph |
|-----------------|--------------------------|---------------|------|-------------------|-----------------|
| Row             | column_name              | data_type     |      |                   |                 |
| 1               | product_id               | INT64         |      |                   |                 |
| 2               | vendor_id                | INT64         |      |                   |                 |
| 3               | market_date              | DATE          |      |                   |                 |
| 4               | customer_id              | INT64         |      |                   |                 |
| 5               | quantity                 | FLOAT64       |      |                   |                 |
| 6               | cost_to_customer_per_qty | FLOAT64       |      |                   |                 |
| 7               | transaction_time         | TIME          |      |                   |                 |

Results per page: 50 1-7 of 7

Job history

## Insights:

The query will return a list of two columns for the **customer\_purchase** table:

- **column\_name**: The name of each column i.e **customer\_id**, **purchase\_date**, **price**.
- **data\_type**: The SQL data type assigned to that column i.e **STRING**, **INTEGER**, **TIMESTAMP**, **FLOAT**.

B. Get the time range between which the orders were placed.

SELECT

```
MIN(transaction_time) AS earliest_transaction_time,
MAX(transaction_time) AS latest_transaction_time
```

FROM

```
`delta-trees-469521-c4`.Target.customer_purchase;
```

Untitled query

```

1 SELECT
2   MIN(transaction_time) AS earliest_transaction_time,
3   MAX(transaction_time) AS latest_transaction_time
4 FROM
5   `delta-trees-469521-c4`.Target.customer_purchase;

```

✓ This query will process 7.84 KB when run.

Query results Save results Open

Job information **Results** Visualisation JSON Execution details Execution graph

| Row | earliest_transacti... | latest_transactio... |
|-----|-----------------------|----------------------|
| 1   | 07:51:00              | 18:59:00             |

Results per page: 50 1 - 1 of 1

### Insights:

- **MIN(transaction\_time)** finds the earliest recorded transaction time. This marks the start of your historical data.
- **MAX(transaction\_time)** finds the latest recorded transaction time. This marks the end of your historical data.

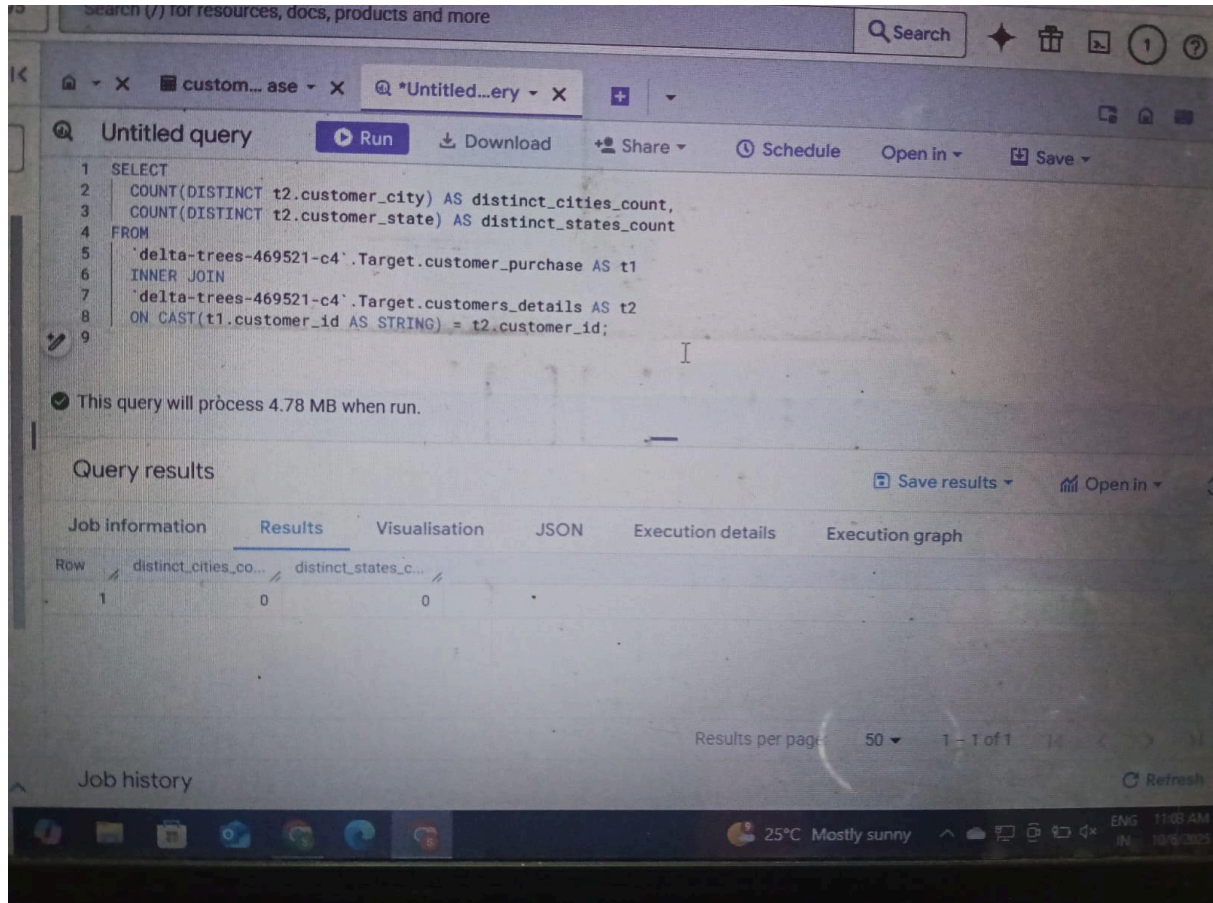
C. Count the Cities & States of customers who ordered during the given period.

```

SELECT
  COUNT(DISTINCT t2.customer_city) AS distinct_cities_count,
  COUNT(DISTINCT t2.customer_state) AS distinct_states_count
FROM
  `delta-trees-469521-c4`.Target.customer_purchase AS t1
  INNER JOIN
  `delta-trees-469521-c4`.Target.customers_details AS t2

```

```
ON CAST(t1.customer_id AS STRING) = t2.customer_id;
```



### Insights:

- **COUNT(DISTINCT t2.customer\_city)**: Calculates the total market breadth at the city level.
- **COUNT(DISTINCT t2.customer\_state)**: Calculates the total market breadth at the state level.
- **customer\_purchase (t1)**: This table is the source of transactions, implying that only customers who have *actually purchased something* are considered.



- `customers_details` (t2): This table contains the customer's geographical information (`customer_city`, `customer_state`).
- The `CAST` function explicitly converts the `customer_id` from the purchase table to a `STRING` so that the join can execute correctly. This is a common practice in data warehousing environments like BigQuery.

## II. In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

```
SELECT
  EXTRACT(YEAR FROM market_date) AS order_year,
  EXTRACT(MONTH FROM market_date) AS order_month,
  COUNT(*) AS number_of_orders
FROM
  `delta-trees-469521-c4`.Target.customer_purchase
GROUP BY order_year, order_month
ORDER BY order_year, order_month;
```

Untitled query

Run Download Share Schedule Open in Save

1 SELECT

This query will process 7.84 KB when run.

Query results

Save results Open in

| Job information | Results    | Visualisation | JSON             | Execution details | Execution graph |
|-----------------|------------|---------------|------------------|-------------------|-----------------|
| Row             | order_year | order_month   | number_of_orders |                   |                 |
| 1               | 2019       | 4             | 67               |                   |                 |
| 2               | 2019       | 5             | 78               |                   |                 |
| 3               | 2019       | 6             | 39               |                   |                 |
| 4               | 2019       | 7             | 134              |                   |                 |
| 5               | 2019       | 8             | 128              |                   |                 |
| 6               | 2019       | 9             | 110              |                   |                 |
| 7               | 2020       | 7             | 167              |                   |                 |
| 8               | 2020       | 8             | 140              |                   |                 |
| 9               | 2020       | 9             | 140              |                   |                 |

Results per page: 50 1 - 9 of 9

Job history Refresh

26°C Mostly sunny ENG 11:26 AM IN 10/6/2022

## Insights:

The primary goal is to generate a simple, time-based metric: the total count of orders per month. This data forms the basis of critical business metrics like:

- **Monthly Sales Velocity:** How fast the business is acquiring orders.
- **Trend Analysis:** Identifying periods of growth, stagnation, or decline.
- **Seasonality:** Spotting recurring peaks (like holidays) and troughs throughout the year.

- **GROUP BY order\_year, order\_month:** This aggregates all the individual transactions into the specific monthly buckets created by the **EXTRACT** functions.
- **COUNT(\*) AS number\_of\_orders:** This counts the number of rows (i.e., the number of orders) within each of the created month/year groups.

B. Can we see some kind of monthly seasonality in terms of the no. of orders being Placed?

```
SELECT
  EXTRACT(MONTH FROM market_date) AS order_month,
  COUNT(*) AS number_of_orders
FROM
  `delta-trees-469521-c4`.Target.customer_purchase
GROUP BY order_month
ORDER BY order_month;
```

Query completed

Query results

Save results Open in

| Job information | Results     | Visualisation    | JSON | Execution details | Execution graph |
|-----------------|-------------|------------------|------|-------------------|-----------------|
| Row             | order_month | number_of_orders |      |                   |                 |
| 1               | 4           | 67               |      |                   |                 |
| 2               | 5           | 78               |      |                   |                 |
| 3               | 6           | 39               |      |                   |                 |
| 4               | 7           | 301              |      |                   |                 |
| 5               | 8           | 268              |      |                   |                 |
| 6               | 9           | 250              |      |                   |                 |

Results per page: 50 1 - 6 of 6 Refresh

Job history

26°C Mostly sunny 11:35 AM 10/10/2025

## Insights:

- **EXTRACT(MONTH FROM market\_date) AS order\_month:** Isolates the month number (1 for Jan, 2 for Feb, etc.).
- **GROUP BY order\_month:** This is the crucial step. By grouping only on the month, the query effectively sums up the total orders for all Januarys, all Februarys, and so on, across the entire period of the data.

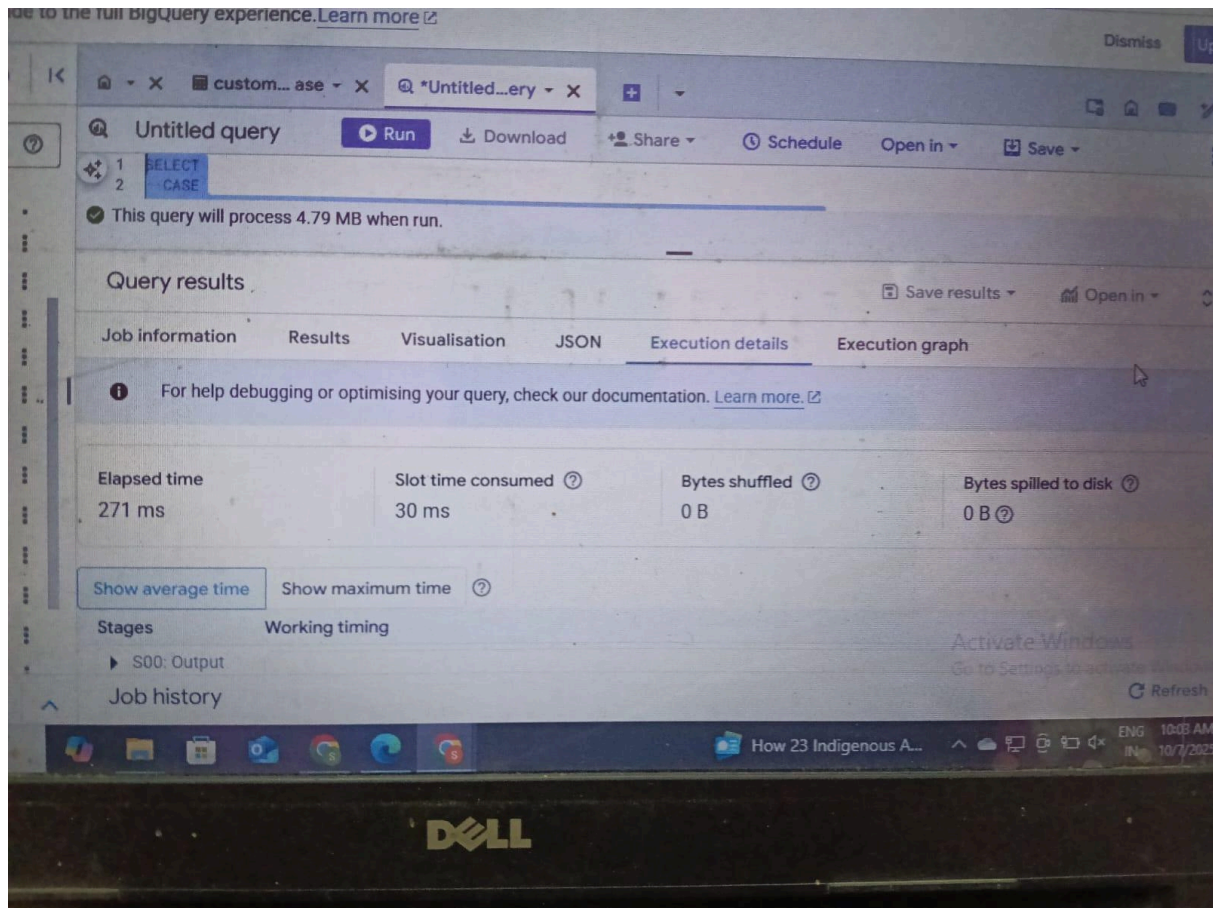
C. During what time of the day, do the Brazilian customers mostly place their



orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
SELECT
  CASE
    WHEN EXTRACT(HOUR FROM t1.transaction_time) BETWEEN 0 AND 6 THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM t1.transaction_time) BETWEEN 7 AND 12 THEN 'Morning'
    WHEN EXTRACT(HOUR FROM t1.transaction_time) BETWEEN 13 AND 18 THEN 'Afternoon'
    WHEN EXTRACT(HOUR FROM t1.transaction_time) BETWEEN 19 AND 23 THEN 'Night'
    ELSE 'Unknown'
  END AS time_of_day,
  COUNT(t1.customer_id) AS number_of_orders
FROM
  `delta-trees-469521-c4`.Target.customer_purchase AS t1
  INNER JOIN
    `delta-trees-469521-c4`.Target.customers_details AS t2
  ON CAST(t1.customer_id AS STRING) = t2.customer_id
WHERE
  t2.customer_state = 'AC' OR t2.customer_city = 'rio branco'
GROUP BY time_of_day
ORDER BY number_of_orders DESC;
```



## Insights:

- **Primary Goal: Region-Specific Time Analysis**
- The main purpose is to uncover the peak ordering behavior (Dawn, Morning, Afternoon, or Night) for customers located in or around the State of Acre (AC).
- The **CASE** statement structures raw timestamp data into business-relevant categories:
- Dawn (0-6 hrs): Very early morning or late night activity.
- Morning (7-12 hrs): Peak commute or work-start activity.
- Afternoon (13-18 hrs): Post-lunch and pre-evening activity.

### III. Evolution of E-commerce orders in the Brazil region:

A. Get the month on month no. of orders placed in each state.

SELECT

EXTRACT(YEAR FROM t1.order\_purchase\_timestamp) AS order\_year,

EXTRACT(MONTH FROM t1.order\_purchase\_timestamp) AS order\_month,

t2.customer\_state,

COUNT(t1.order\_id) AS number\_of\_orders

FROM

`delta-trees-469521-c4`.Target.orders AS t1

INNER JOIN

`delta-trees-469521-c4`.Target.`customers ` AS t2

ON t1.customer\_id = t2.customer\_id

GROUP BY order\_year, order\_month, t2.customer\_state

ORDER BY order\_year, order\_month, t2.customer\_state;





- The primary insight provided by this query is a Month-on-Month E-commerce Order Volume Breakdown by State.

B. How are the customers distributed across all the states?

SELECT

t1.customer\_state,

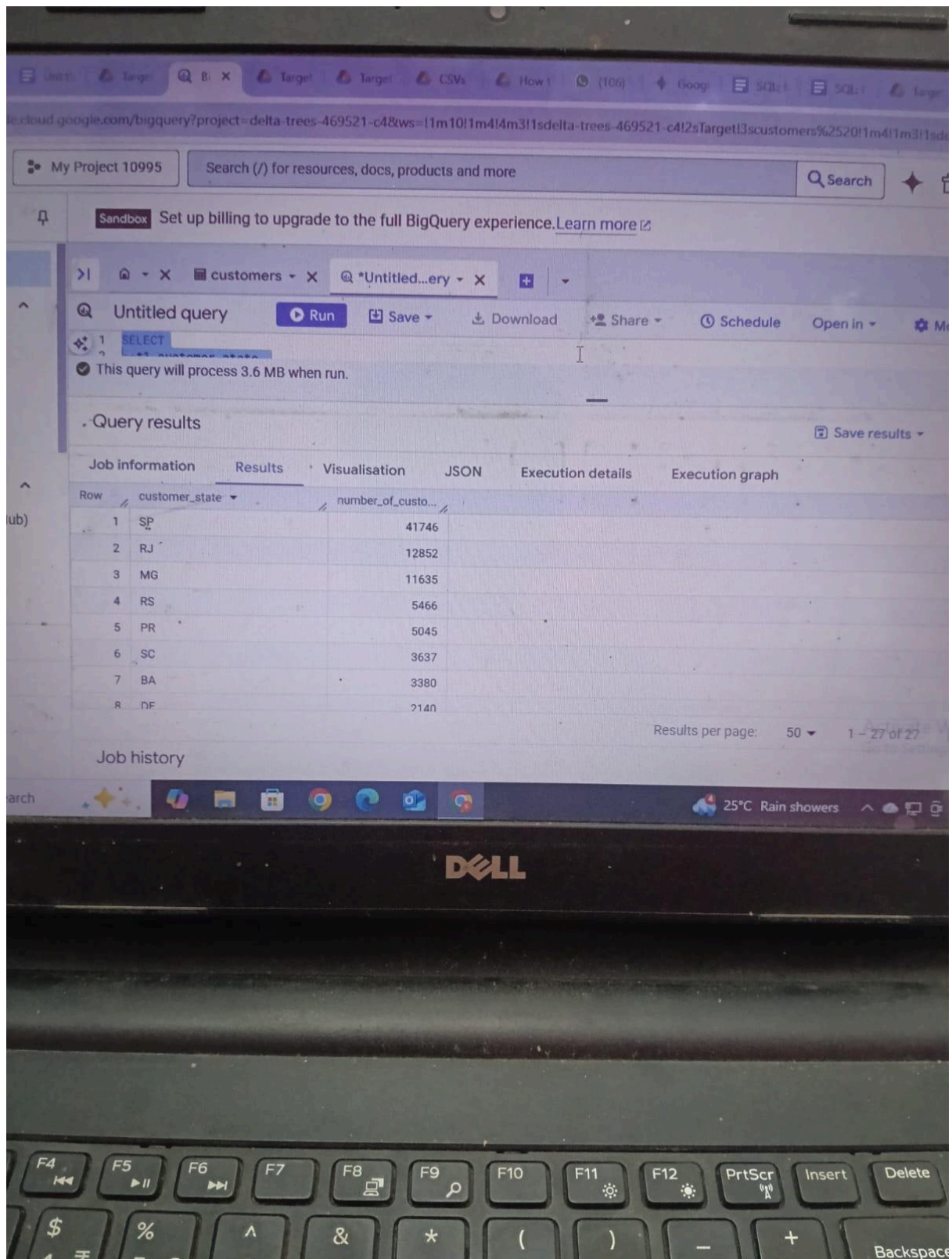
COUNT(t1.customer\_id) AS number\_of\_customers

FROM

`delta-trees-469521-c4`.Target.`customers` AS t1

GROUP BY t1.customer\_state

ORDER BY number\_of\_customers DESC;



**Insights:**

The primary insight provided by this query is the distribution of your customer base across different states in Brazil.

The output will be a sorted list that immediately reveals your company's most important geographic markets based on the absolute number of customers.

IV. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

WITH

YearlyOrderCosts AS (

SELECT

EXTRACT(YEAR FROM t1.order\_purchase\_timestamp) AS  
order\_year,

SUM(t2.price + t2.freight\_value) AS total\_order\_cost

FROM

`delta-trees-469521-c4`.Target.orders AS t1

INNER JOIN

`delta-trees-469521-c4`.Target.order\_items AS t2

ON t1.order\_id = t2.order\_id

WHERE

EXTRACT(YEAR FROM t1.order\_purchase\_timestamp) IN (2017,  
2018) AND EXTRACT(MONTH FROM t1.order\_purchase\_timestamp)  
BETWEEN 1 AND

8

GROUP BY order\_year

),

Costs AS (

SELECT

(

SELECT

total\_order\_cost

FROM

YearlyOrderCosts

WHERE

order\_year = 2017

) AS cost\_2017,

(

SELECT



**total\_order\_cost**

**FROM**

**YearlyOrderCosts**

**WHERE**

**order\_year = 2018**

**) AS cost\_2018**

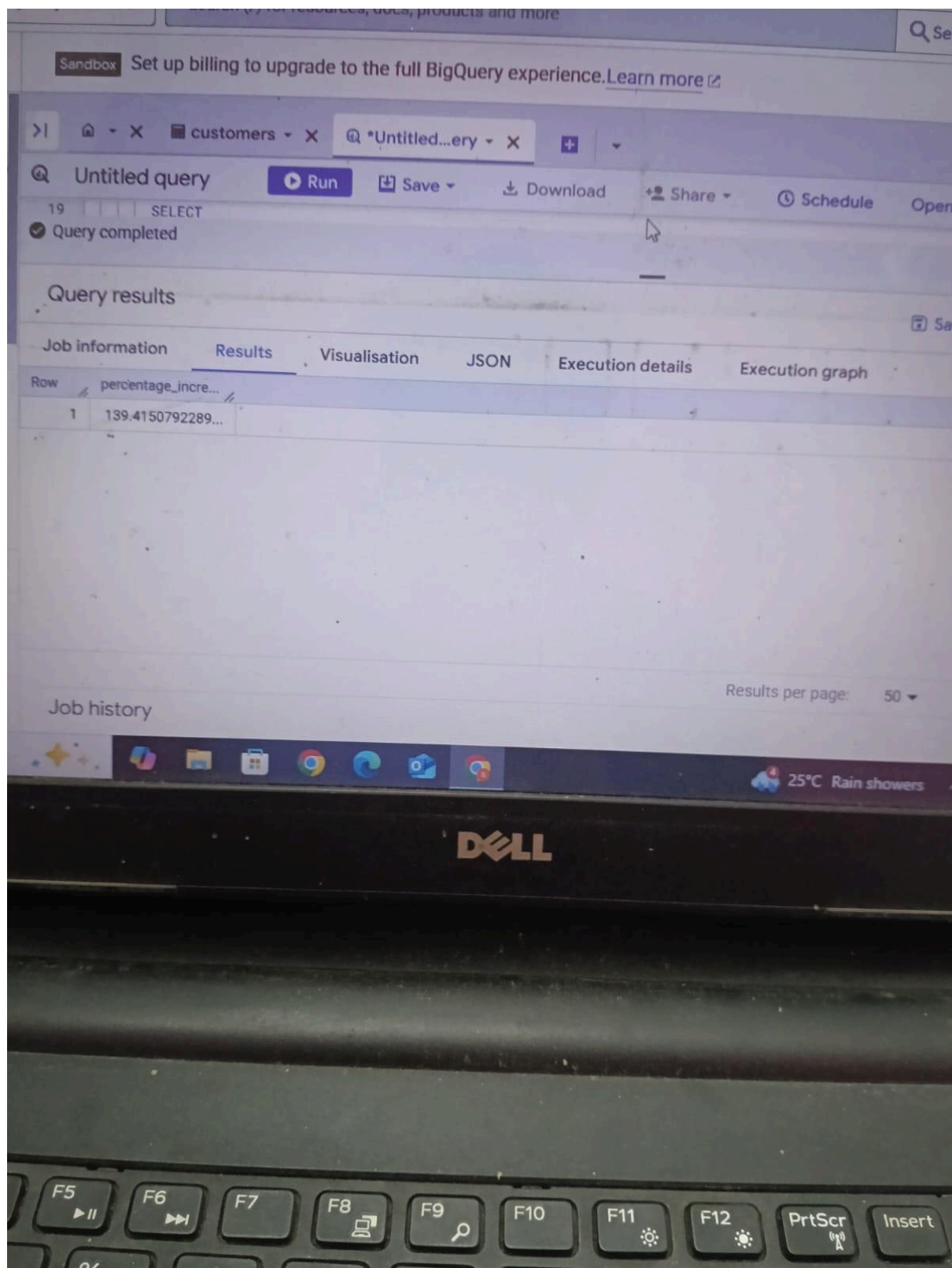
**)**

**SELECT**

**((cost\_2018 - cost\_2017) / cost\_2017) \* 100 AS  
percentage\_increase**

**FROM**

**Costs;**

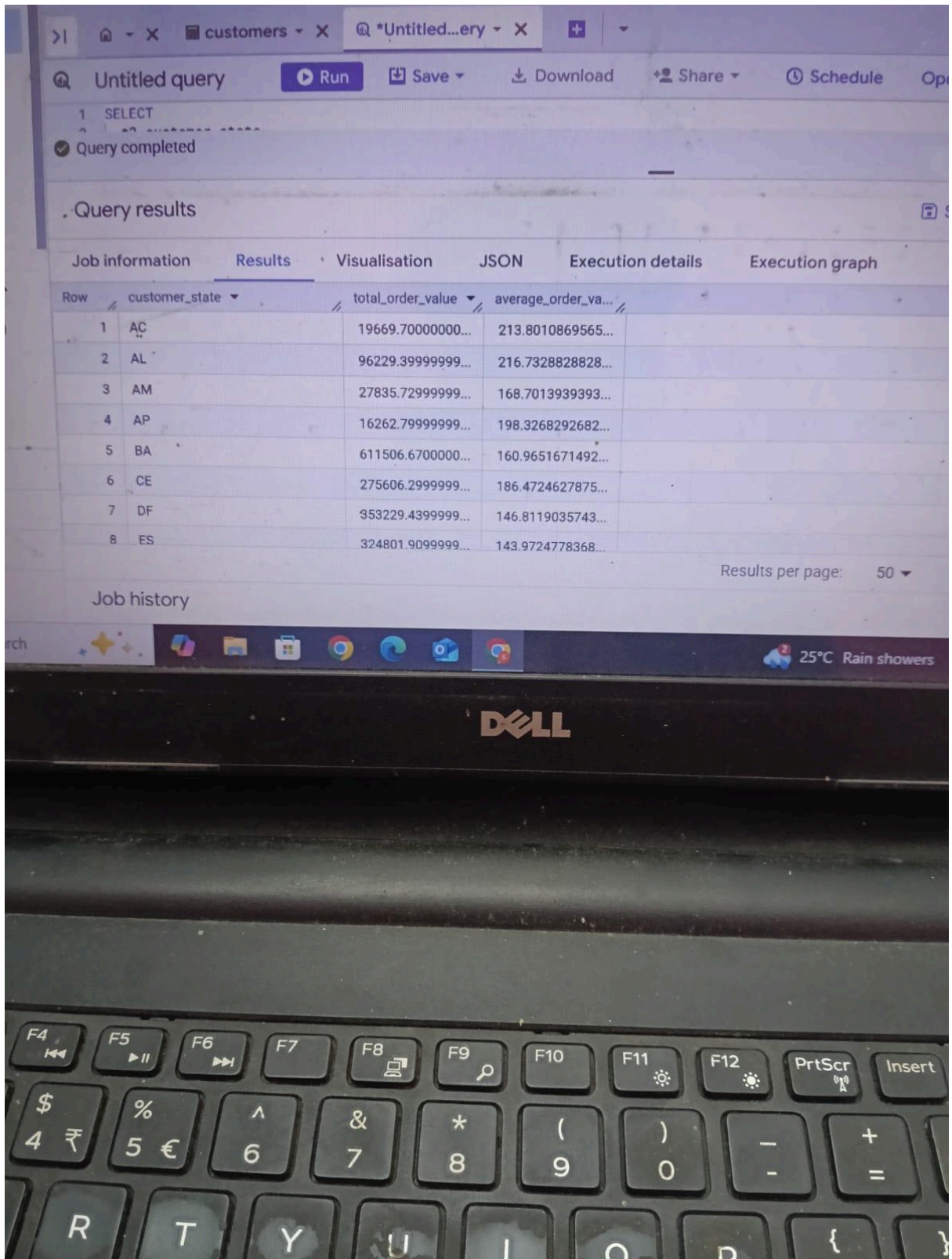


**Insights:**

- The primary insight derived from this query is the growth rate of total e-commerce expenditure over a fixed, specific period.

B. Calculate the Total & Average value of order price for each state.

```
SELECT
    t3.customer_state,
    SUM(t2.price + t2.freight_value) AS total_order_value,
    AVG(t2.price + t2.freight_value) AS average_order_value
FROM
    `delta-trees-469521-c4`.Target.orders AS t1
    INNER JOIN
        `delta-trees-469521-c4`.Target.order_items AS t2
    ON t1.order_id = t2.order_id
    INNER JOIN
        `delta-trees-469521-c4`.Target.`customers ` AS t3
    ON t1.customer_id = t3.customer_id
GROUP BY t3.customer_state
ORDER BY t3.customer_state;
```



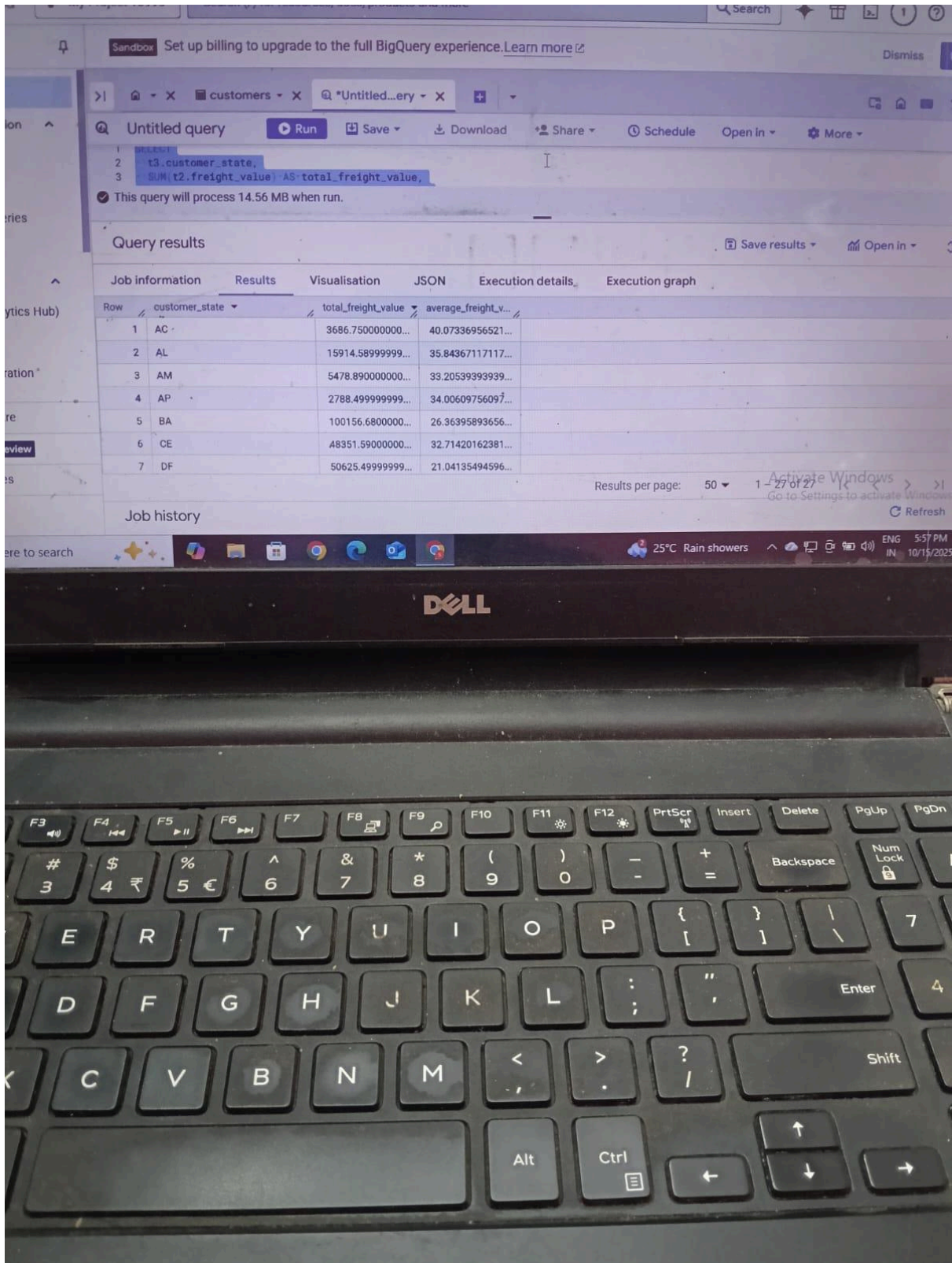
## Insights:

- The primary insight provided is a detailed breakdown of which Brazilian states generate the most total revenue and have the highest average customer spend, thereby identifying the most valuable and profitable geographic markets.

C. Calculate the Total & Average value of order freight for each state.

```
SELECT
    t3.customer_state,
    SUM(t2.freight_value) AS total_freight_value,
    AVG(t2.freight_value) AS average_freight_value
FROM
    `delta-trees-469521-c4`.Target.orders AS t1
    INNER JOIN
    `delta-trees-469521-c4`.Target.order_items AS t2
    ON t1.order_id = t2.order_id
    INNER JOIN
    `delta-trees-469521-c4`.Target.`customers ` AS t3
    ON t1.customer_id = t3.customer_id
GROUP BY t3.customer_state
ORDER BY t3.customer_state;
```





Insights:

- This query is a crucial tool for logistical planning and cost management by breaking down shipping expenses across different geographical regions.
- The primary insight provided by this query is the freight cost profile for each Brazilian state. It helps identify logistical inefficiencies and areas where shipping costs significantly impact profitability.

## V. Analysis based on sales, freight and delivery time.

A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

```
SELECT
    order_id,
    DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS
delivery_time_days,
    DATE_DIFF(order_estimated_delivery_date, order_delivered_customer_date, DAY) AS
estimated_vs_actual_delivery_diff_days
FROM
    `delta-trees-469521-c4`.Target.orders
WHERE
    order_delivered_customer_date IS NOT NULL AND order_purchase_timestamp IS NOT
NULL AND order_estimated_delivery_date IS NOT NULL;
```

cloud.google.com/bigquery?project=delta-trees-469521-c48&ws=11m1011m414m31delta-trees-469521-c412sTarget3scustomers%25201m411m31delta-tree...

My Project 10995 Search (/) for resources, docs, products and more

Sandbox Set up billing to upgrade to the full BigQuery experience. [Learn more](#)

Untitled query Run Save Download Share Schedule Open in More

```
1 SELECT
2   order_id,
3   DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS delivery_time_days,
```

This query will process 5.48 MB when run.

Query results Save results Open

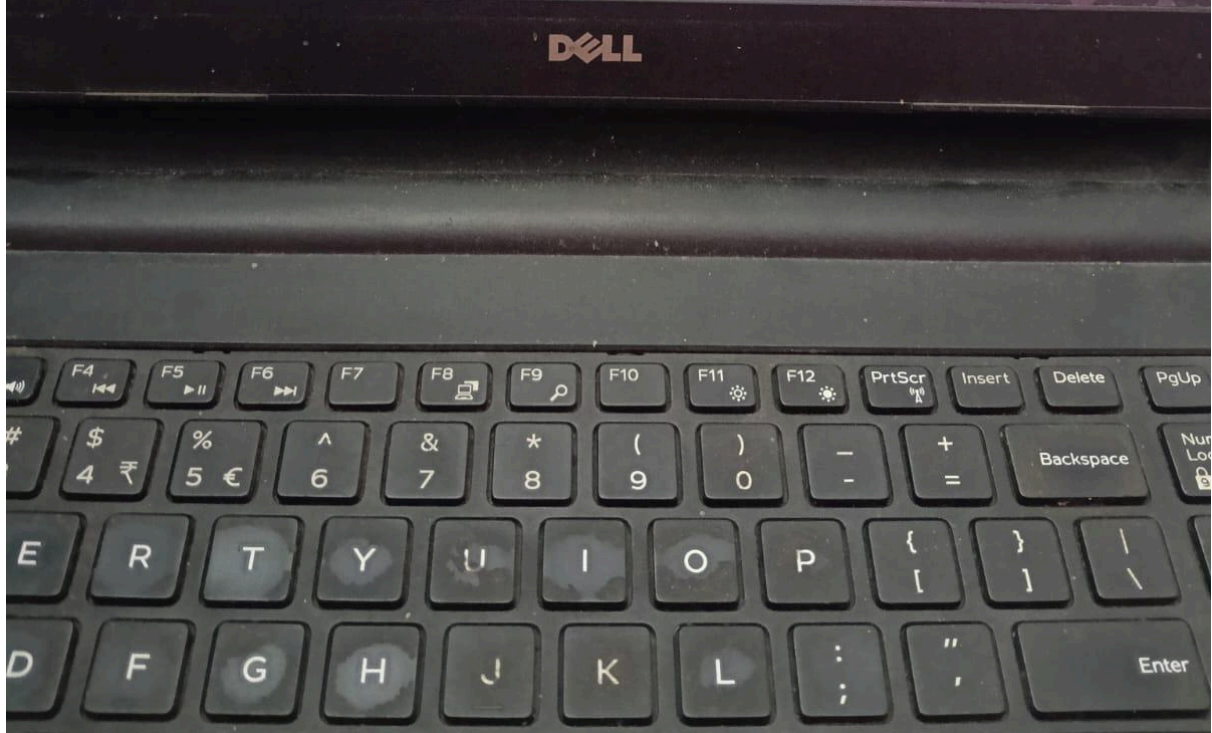
Job information Results Visualisation JSON Execution details Execution graph

| Row | order_id                       | delivery_time_days | estimated_vs_act... |
|-----|--------------------------------|--------------------|---------------------|
| 1   | 65d1e226dfaeb8cd42f665422...   | 35                 | 16                  |
| 2   | 2c45c33d2f9cb8ff8b1c86cc28...  | 30                 | 28                  |
| 3   | 1950d777989f6a877539f53795...  | 30                 | -12                 |
| 4   | bfbdf0f9bdef84302105ad712db... | 54                 | -36                 |
| 5   | 98974b076b01553d49ee64679...   | 43                 | 6                   |
| 6   | c4b41c36dd589e901f6879f25a...  | 36                 | 14                  |
| 7   | d2289ff2201a74a5db154d11b7a... | 20                 | 20                  |

Results per page: 50 1 - 50 of 96476

Job history

search 25°C Rain showers



## Insights:

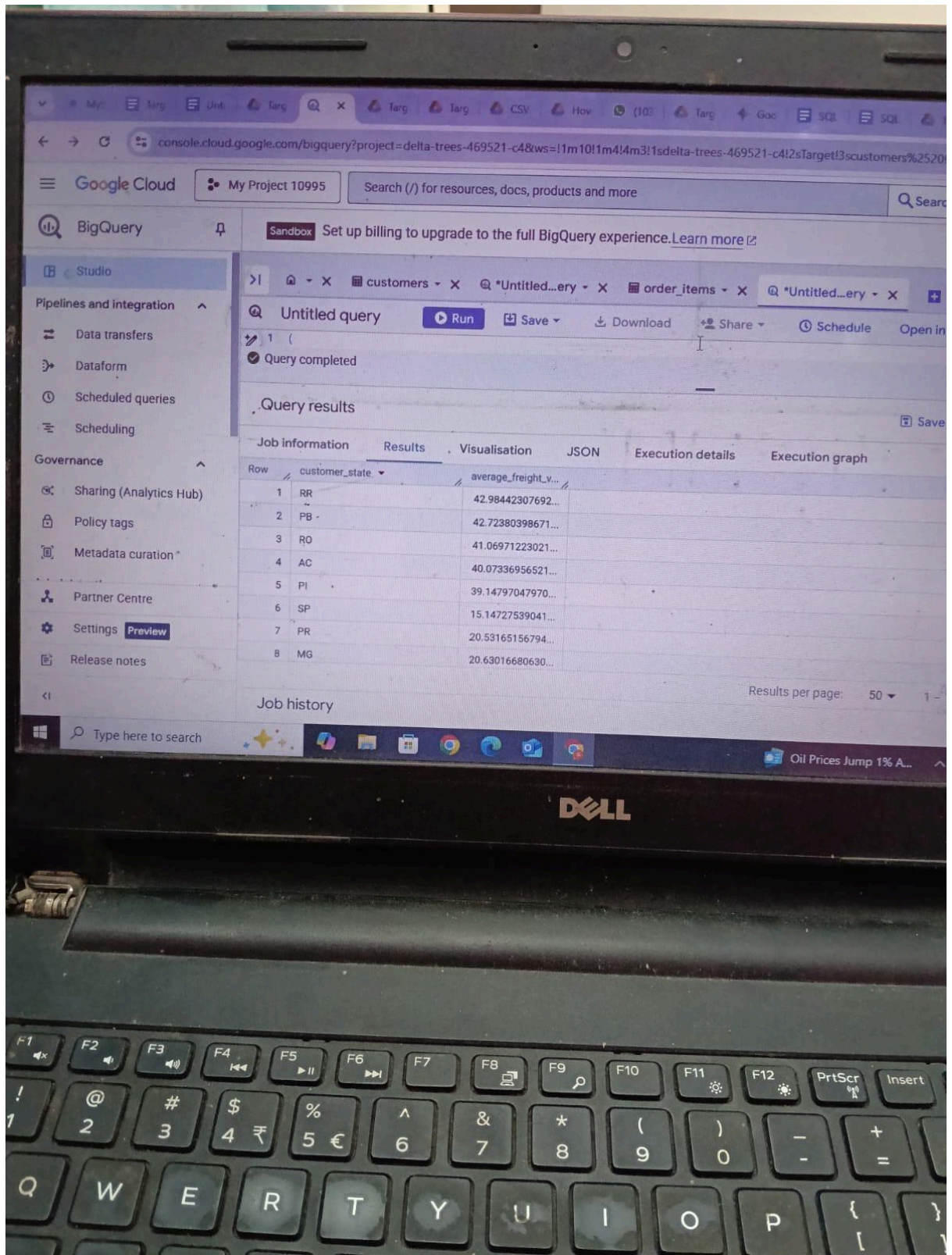
- The primary insight provided is an order-by-order report that quantifies two critical metrics for logistics and customer experience: actual delivery time and prediction accuracy.

B. Find out the top 5 states with the highest & lowest average freight value.

```
(  
SELECT  
    t3.customer_state,  
    AVG(t2.freight_value) AS average_freight_value  
FROM  
    `delta-trees-469521-c4`.Target.orders AS t1  
    INNER JOIN  
    `delta-trees-469521-c4`.Target.order_items AS t2  
    ON t1.order_id = t2.order_id  
    INNER JOIN  
    `delta-trees-469521-c4`.Target.`customers ` AS t3  
    ON t1.customer_id = t3.customer_id  
GROUP BY t3.customer_state  
ORDER BY average_freight_value DESC  
LIMIT 5) UNION ALL(  
SELECT  
    t3.customer_state,  
    AVG(t2.freight_value) AS average_freight_value  
FROM
```

```
`delta-trees-469521-c4`.Target.orders AS t1
INNER JOIN
`delta-trees-469521-c4`.Target.order_items AS t2
ON t1.order_id = t2.order_id
INNER JOIN
`delta-trees-469521-c4`.Target.`customers ` AS t3
ON t1.customer_id = t3.customer_id
GROUP BY t3.customer_state
ORDER BY average_freight_value
LIMIT 5);
```





Insights:

- The query combines two identical subqueries using **UNION ALL** to create a single report that clearly highlights the top 5 most expensive states to ship to and the bottom 5 least expensive states to ship to, based on the Average Freight Value (AOV) per item.

C. Find out the top 5 states with the highest & lowest average delivery time.

(

SELECT

t2.customer\_state,

AVG(TIMESTAMP\_DIFF(t1.order\_delivered\_customer\_date,  
t1.order\_purchase\_timestamp, SECOND)) AS  
average\_delivery\_time\_seconds

FROM

`delta-trees-469521-c4`.Target.orders AS t1

INNER JOIN

`delta-trees-469521-c4`.Target.`customers ` AS t2

ON t1.customer\_id = t2.customer\_id

WHERE

t1.order\_delivered\_customer\_date IS NOT NULL AND  
t1.order\_purchase\_timestamp IS NOT NULL

GROUP BY t2.customer\_state

```
ORDER BY average_delivery_time_seconds DESC

LIMIT 5) UNION ALL(

SELECT

    t2.customer_state,

    AVG(TIMESTAMP_DIFF(t1.order_delivered_customer_date,
t1.order_purchase_timestamp, SECOND)) AS
average_delivery_time_seconds

FROM

    `delta-trees-469521-c4`.Target.orders AS t1

INNER JOIN

    `delta-trees-469521-c4`.Target.`customers ` AS t2

ON t1.customer_id = t2.customer_id

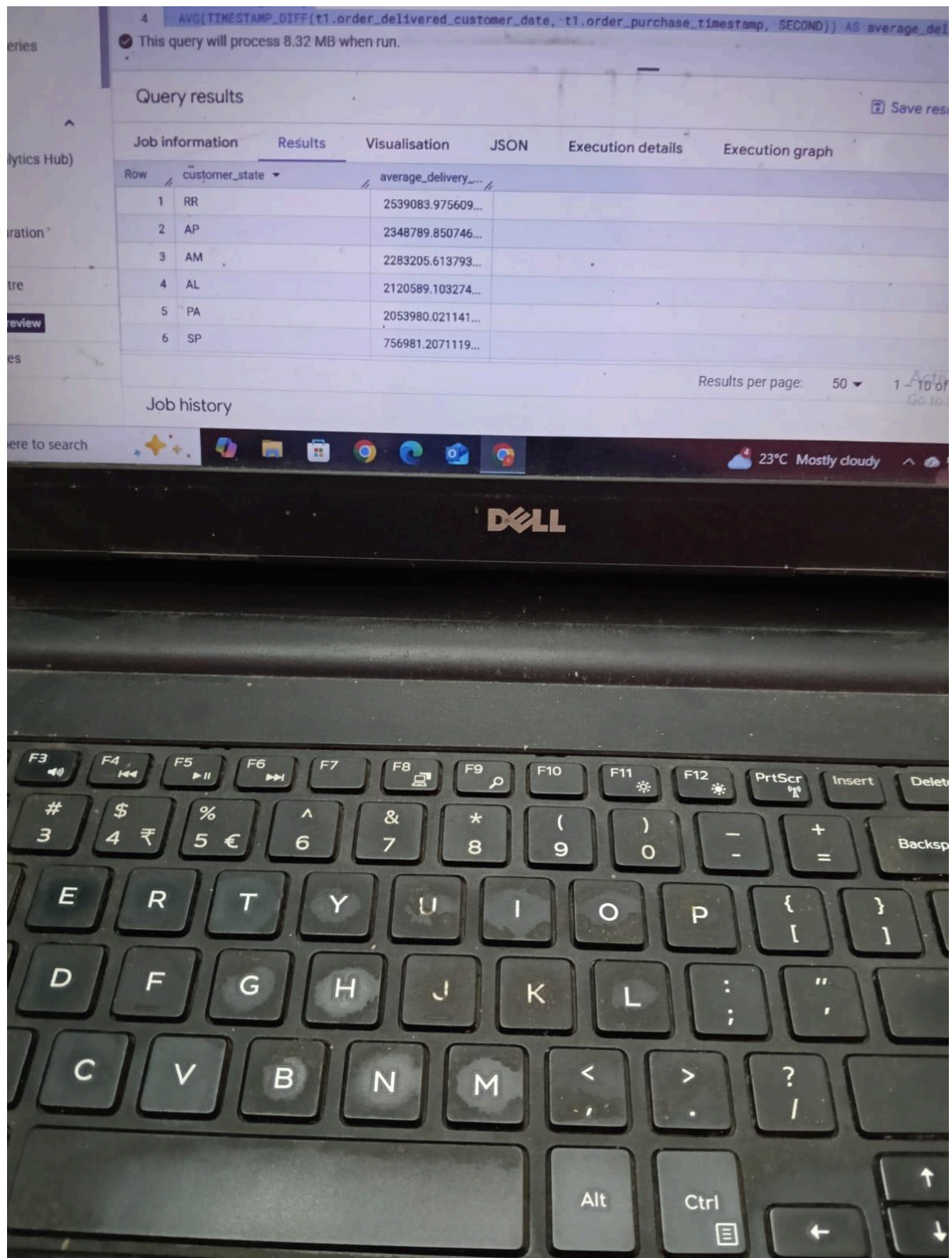
WHERE

    t1.order_delivered_customer_date IS NOT NULL AND
t1.order_purchase_timestamp IS NOT NULL

GROUP BY t2.customer_state

ORDER BY average_delivery_time_seconds

LIMIT 5);
```



Insights:



- The query uses a **UNION ALL** structure to produce a single, high-impact report that identifies the top 5 slowest states for delivery and the top 5 fastest states for delivery, based on the average time elapsed between purchase and customer delivery.

D. Find out the top 5 states where the order delivery is really fast as compared to

the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery

date to figure out how fast the delivery was for each state.

**SELECT**

t2.customer\_state,

AVG(TIMESTAMP\_DIFF(t1.order\_estimated\_delivery\_date,  
t1.order\_delivered\_customer\_date, SECOND)) AS average\_delivery\_speed\_seconds

**FROM**

`delta-trees-469521-c4`.Target.orders AS t1

**INNER JOIN**

`delta-trees-469521-c4`.Target.customers AS t2

**ON** t1.customer\_id = t2.customer\_id

**WHERE**

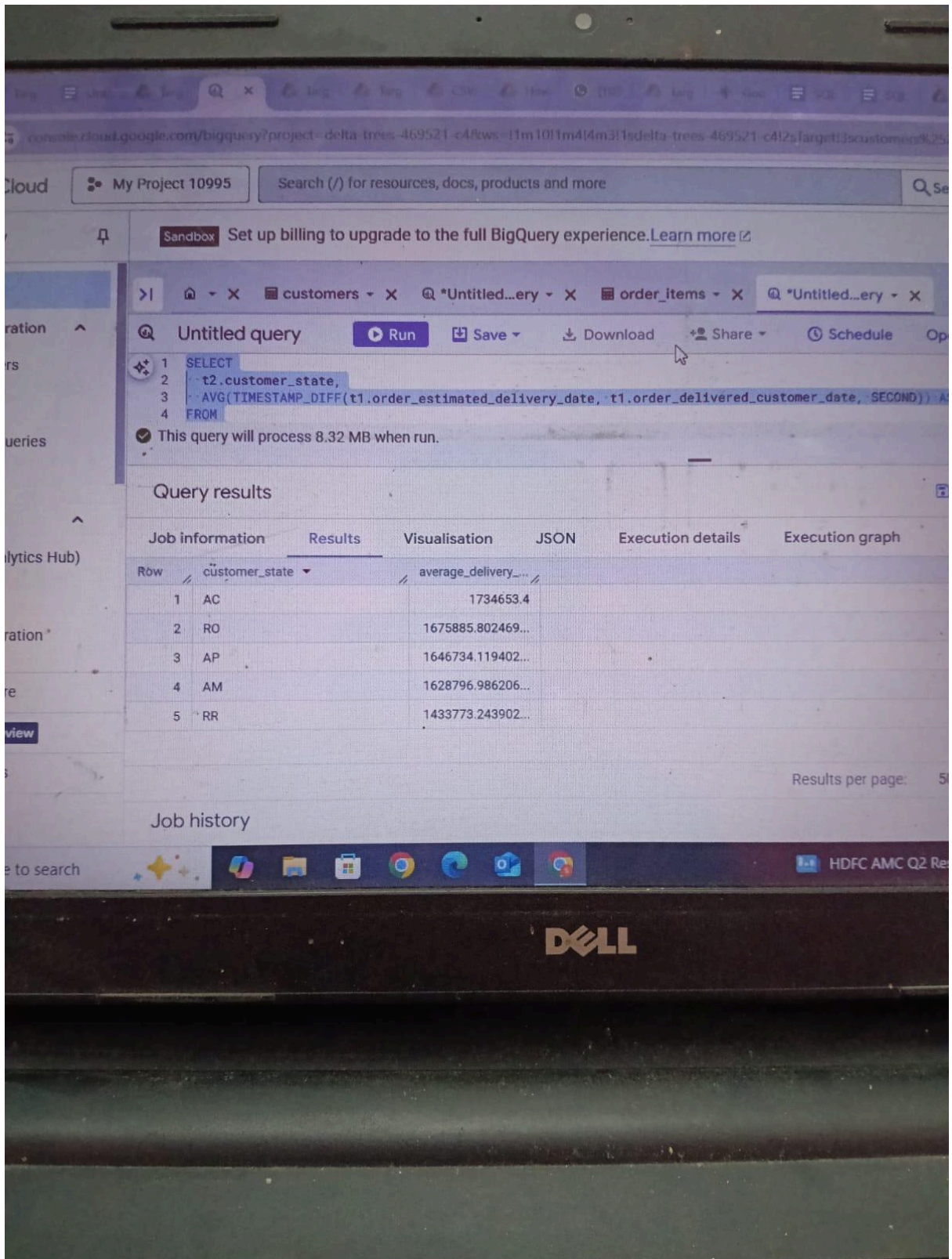
t1.order\_delivered\_customer\_date IS NOT NULL AND t1.order\_estimated\_delivery\_date  
IS NOT NULL

**GROUP BY** t2.customer\_state



```
ORDER BY average_delivery_speed_seconds DESC
```

```
LIMIT 5;
```



```
1 SELECT
2   t2.customer_state,
3   AVG(TIMESTAMP_DIFF(t1.order_estimated_delivery_date, t1.order_delivered_customer_date, SECOND)) AS
4 FROM
```

✓ This query will process 8.32 MB when run.

### Query results

| Job information |                | Results             | Visualisation | JSON | Execution details | Execution graph |
|-----------------|----------------|---------------------|---------------|------|-------------------|-----------------|
| Row             | customer_state | average_delivery... |               |      |                   |                 |
| 1               | AC             | 1734653.4           |               |      |                   |                 |
| 2               | RO             | 1675885.802469...   |               |      |                   |                 |
| 3               | AP             | 1646734.119402...   |               |      |                   |                 |
| 4               | AM             | 1628796.986206...   |               |      |                   |                 |
| 5               | RR             | 1433773.243902...   |               |      |                   |                 |

Results per page: 5

### Job history

## Insights:

- The primary insight provided is the identification of the top 5 states with the greatest positive delivery variance, meaning the states where deliveries arrive the earliest relative to the estimated date.

## VI. Analysis based on the payments:

A. Find the month on month no. of orders placed using different payment types.

SELECT

EXTRACT(YEAR FROM t1.order\_purchase\_timestamp) AS  
order\_year,

EXTRACT(MONTH FROM t1.order\_purchase\_timestamp) AS  
order\_month,

t2.payment\_type,

COUNT(t1.order\_id) AS number\_of\_orders

FROM

`delta-trees-469521-c4`.Target.orders AS t1

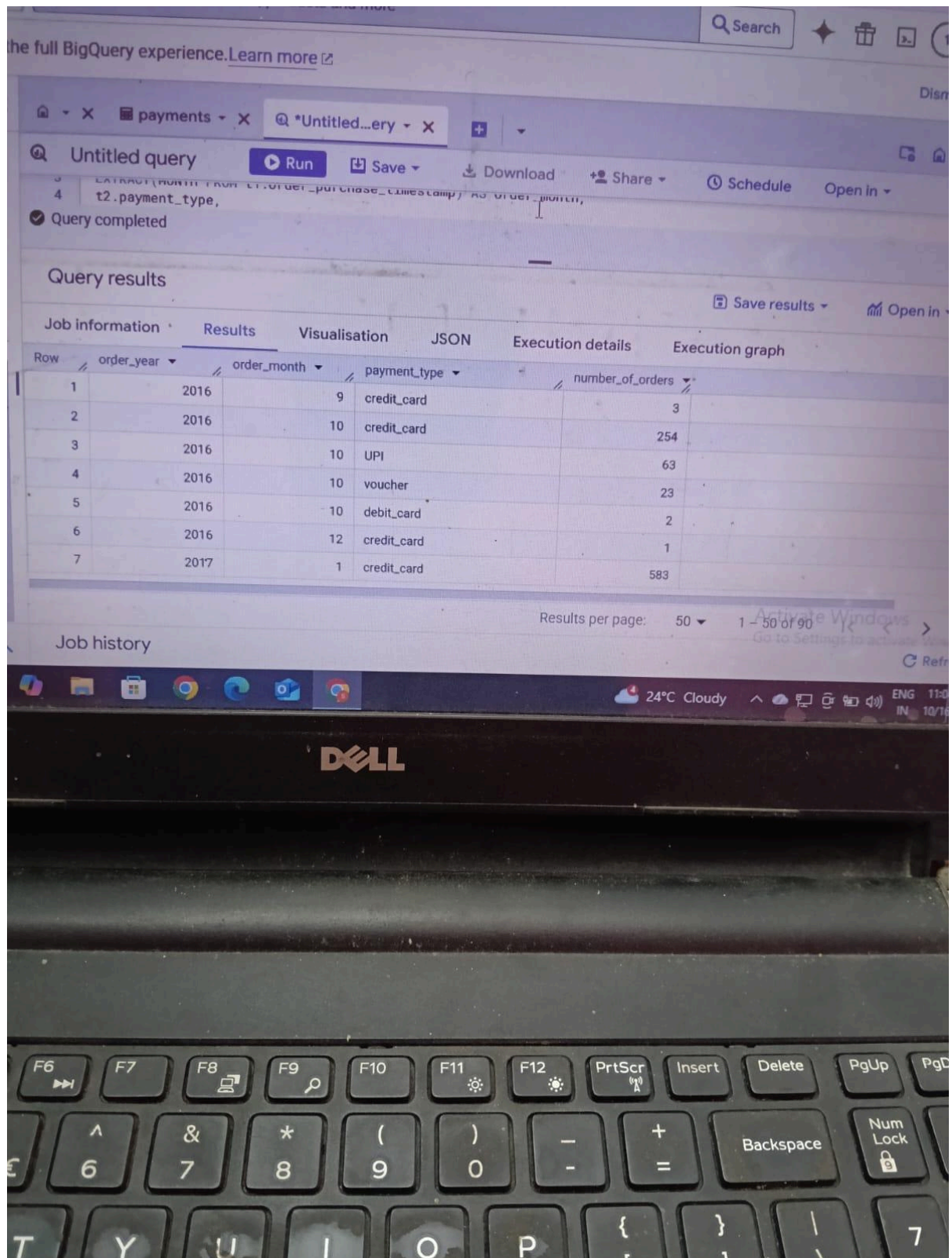
INNER JOIN

`delta-trees-469521-c4`.Target.payments AS t2

ON t1.order\_id = t2.order\_id

**GROUP BY order\_year, order\_month, t2.payment\_type**

**ORDER BY order\_year, order\_month, number\_of\_orders DESC;**



Insights:



- The primary insight is the **Monthly Evolution of Payment Method Popularity**. It generates a detailed report that shows the volume of orders processed for each `payment_type` (e.g., credit card, debit card, boleto) over time.

B. Find the no. of orders placed on the basis of the payment installments that have been paid.

SELECT

t2.payment\_installments,

COUNT(t1.order\_id) AS number\_of\_orders

FROM

`delta-trees-469521-c4`.Target.orders AS t1

INNER JOIN

`delta-trees-469521-c4`.Target.payments AS t2

ON t1.order\_id = t2.order\_id

GROUP BY t2.payment\_installments

ORDER BY t2.payment\_installments;

untitled...google.com/bigquery/project-delta-trees-469521-c48cws-11m101m414m311delta-trees-469521-c412sTarget13payments11m411m311a

My Project 10995 Search (/) for resources, docs, products and more Search

illing to upgrade to the full BigQuery experience. [Learn more](#)

+ Add data |< payments \*Untitled...ery +

resources ?

Untitled query Run Save Download Share Schedule

1 SELECT  
Query completed

Query results Save results

Job information Results Visualisation JSON Execution details Execution graph

| Row | payment_installm... | number_of_orders |
|-----|---------------------|------------------|
| 1   | 0                   | 2                |
| 2   | 1                   | 52546            |
| 3   | 2                   | 12413            |
| 4   | 3                   | 10461            |
| 5   | 4                   | 7098             |
| 6   | 5                   | 5239             |
| 7   | 6                   | 3920             |
| 8   | 7                   | 1626             |

Results per page: 50 1 - 24

Job history

24°C Cloudy

DELL

F4 F5 F6 F7 F8 F9 F10 F11 F12 PrtScr Insert  
\$ % ^ & \* ( ) - +

Insights:

- The primary insight provided is the **distribution of orders across different installment options**. It reveals the customer's reliance on credit and the corresponding financial risk exposure for the business.